

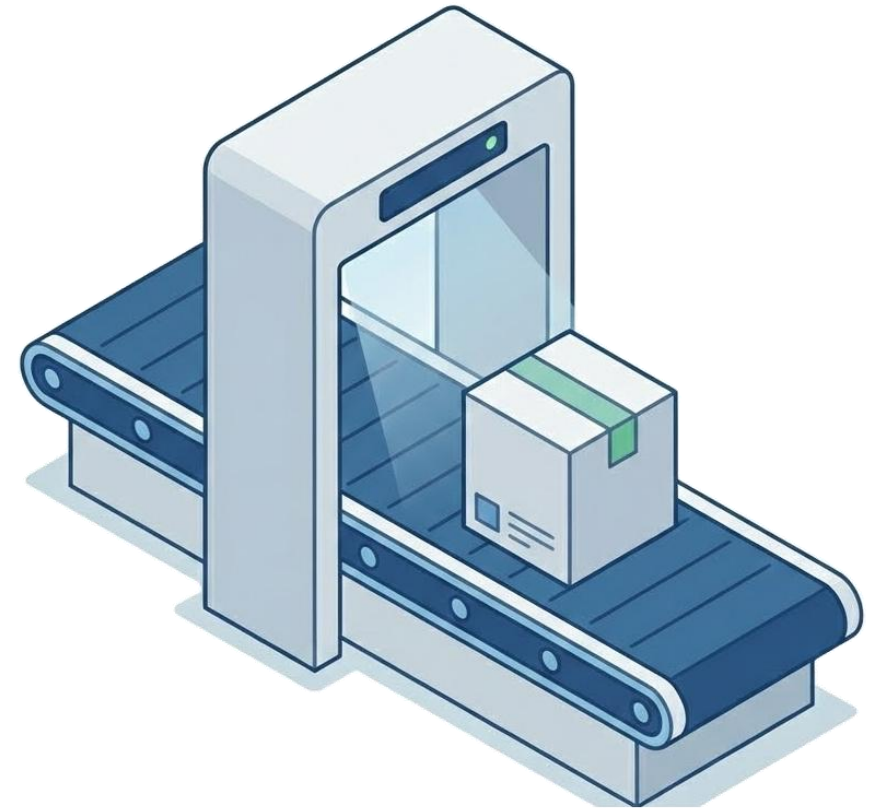
IngressX 제품소개서

(주)테크웨어 2026. 02.

오픈소스 신청-검사-승인-반입 플랫폼

IngressX

Public Repository 직접 사용을 차단하고,
승인된 오픈소스만 Internal Repository에 반입하여 사용하게 합니다.



IngressX

오픈소스 신청-검사-승인-반입 플랫폼

오픈소스 반입 신청 및 검사 후 승인된 오픈소스만 내부 Repository에서 안전하게 사용합니다

제품명 : IngressX

개발사 : (주)테크웨어

외부 차단 (통제)



외부 Public Repository 직접 연결 차단

반입 프로세스



"의존성 선언 파일" 업로드 기반으로 반입
요청후 다운로드/검사/승인 워크플로우 운영

내부 자산화



승인된 오픈소스를 Internal Repository에
저장하여 조직 표준 Repository로 운영



Public Repository



IngressX



Internal Repository



개발 환경

지원 생태계 | Maven npm Gradle pip

검사 도구 |



+





상용 SW 포함 비율

97%

상용 소프트웨어 중
오픈소스 포함 비율

출처*1 : Synopsys OSSRA 2025



평균 의존성 수

150+

평균 Java 애플리케이션의
오픈소스 의존성 수

출처*2 : Sonatype 2024



전자정부 표준프레임워크 구성

수십 개

전자정부 표준프레임워크
포함 오픈소스 컴포넌트

출처*3 : NIA 표준프레임워크 포털

“

오픈소스는 선택이 아닌 필수입니다.

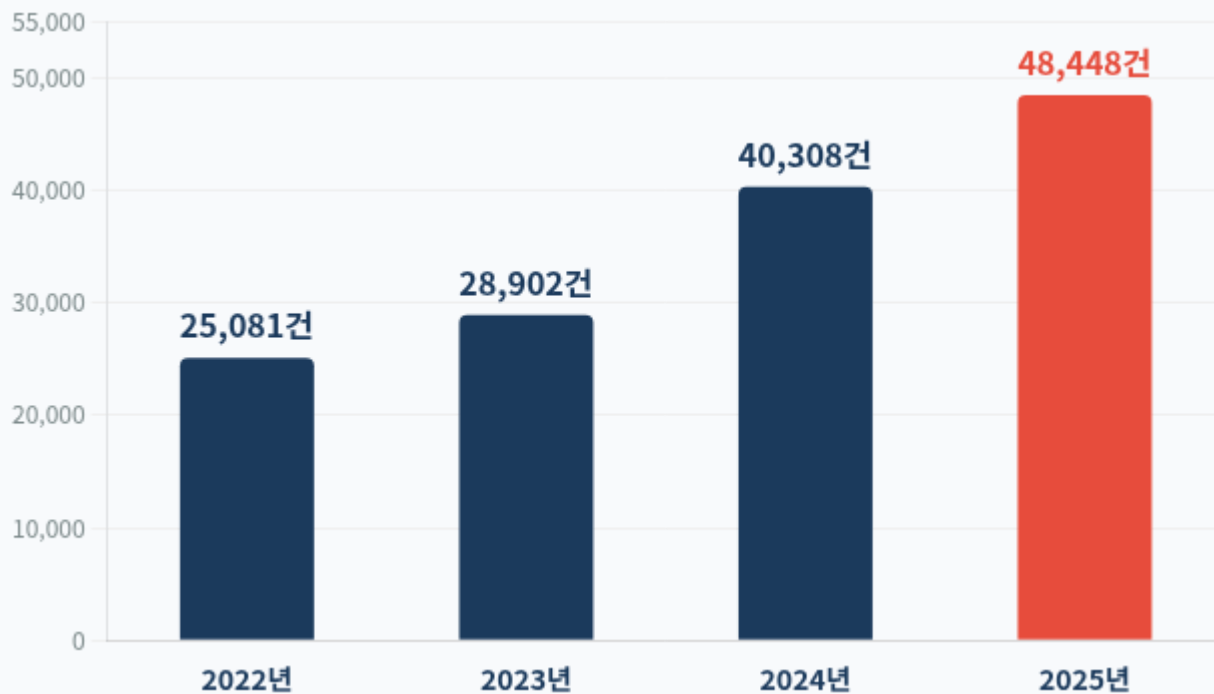
문제는 "사용 여부"가 아니라, "어떻게 관리하는가" 입니다.

출처*1 : <https://www.blackduck.com/blog/open-source-trends-ossra-report.html#2>

출처*2 : <https://www.sonatype.com/state-of-the-software-supply-chain/2024/scale>

출처*3 : <https://www.egovframe.go.kr/home/sub.do?menuNo=13>

트 CVE 발견 건수 추이



출처*1 : CVE Details

출처*1 : <https://www.cvedetails.com/browse-by-date.php>

출처*2 : https://www.sonatype.com/hubfs/SSCR-2024/SSCR_2024-FINAL-10-10-24.pdf

출처*3 : https://www.sonatype.com/hubfs/White_Papers/2024-in-Open-Source-Malware-Report.pdf

2024년 악성 패키지 발견

512,847 건

↑ 전년 대비 156% 증가

출처*2 : sonatype

2019년 이후 누적 악성 패키지

778,529 건

출처*3 : sonatype

⚠ 위협의 고도화

매년 수만 건의 새로운 취약점이 발견되고 있으며,
악의적인 **공급망 공격**은 **기하급수적으로 증가**하고 있습니다.

3대 “Shell” 취약점

2021	2022	2025
Log4Shell 2021.12 CVE-2021-44228 CVSS 10.0 (최고등급) Apache Log4j 원격 코드 실행(RCE) 취약점 Amazon, Google, Microsoft 등 주요 클라우드 서비스 포함 전 세계 수억 대의 시스템에 영향	Spring4Shell 2022.03 CVE-2022-22965 CVSS 9.8 Spring Framework 원격 코드 실행(RCE) 취약점 전자정부 표준프레임워크 핵심 Spring에서 발생 엔터프라이즈 Java 환경에 광범위 영향	React2Shell 2025.12 CVE-2025-55182 CVSS 10.0 (최고등급) React Server Components 원격 코드 실행(RCE) 취약점 업스트림 취약점이 다운스트림 프레임워크에 연쇄 영향. 현대 React 기반 웹 프레임워크 생태계 전반에 영향

“

오픈소스 구성요소의 취약점은 개별 기업의 문제가 아니라 **생태계 구조의 문제**이며, 해당 구성요소를 사용하는 환경 전반에 영향을 미칩니다.

오픈소스는 어떻게 반입하여 사용되고 있습니까?

소스에 수동 직접 포함



✓ 자동화 없음 ✓ 통제 어려움

Public Repository 직접 사용



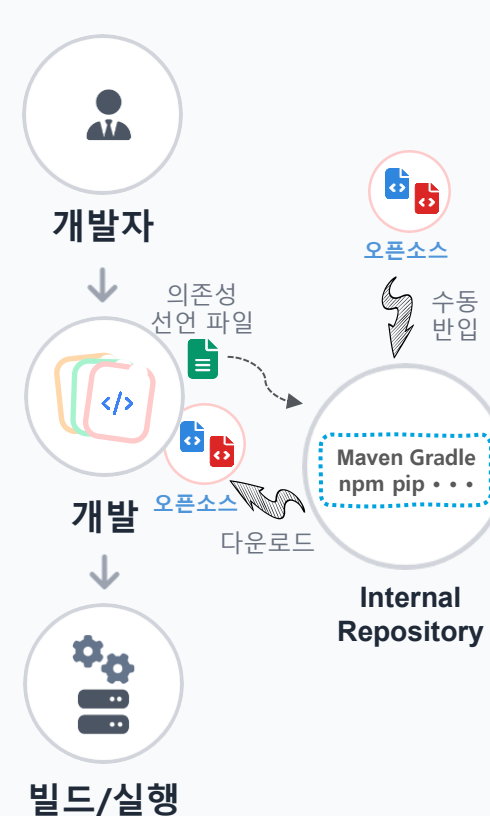
✓ 자동화 가능 ✓ 통제 어려움

Internal Repository 자동 연계



✓ 자동화 가능 ✓ 통제 어려움

Internal Repository + 수동 반입



✓ 자동화 가능 ✓ 통제 어려움

오픈소스 사용은 늘어나지만, 관리 체계는 따라가지 못하고 있습니다

1 사후 발견, 사후 대응

개발 중/후 취약점 발견



이미 코드베이스에 침투

2 수동 프로세스

개발자가 직접 Public Repository 접근



검사 없이 다운로드

3 내부 Repository 미관리

검사 및 승인되지 않은 오픈소스 혼재



안전 여부 불명확

4 표준화 부재

팀/프로젝트마다 다른 검사 방식



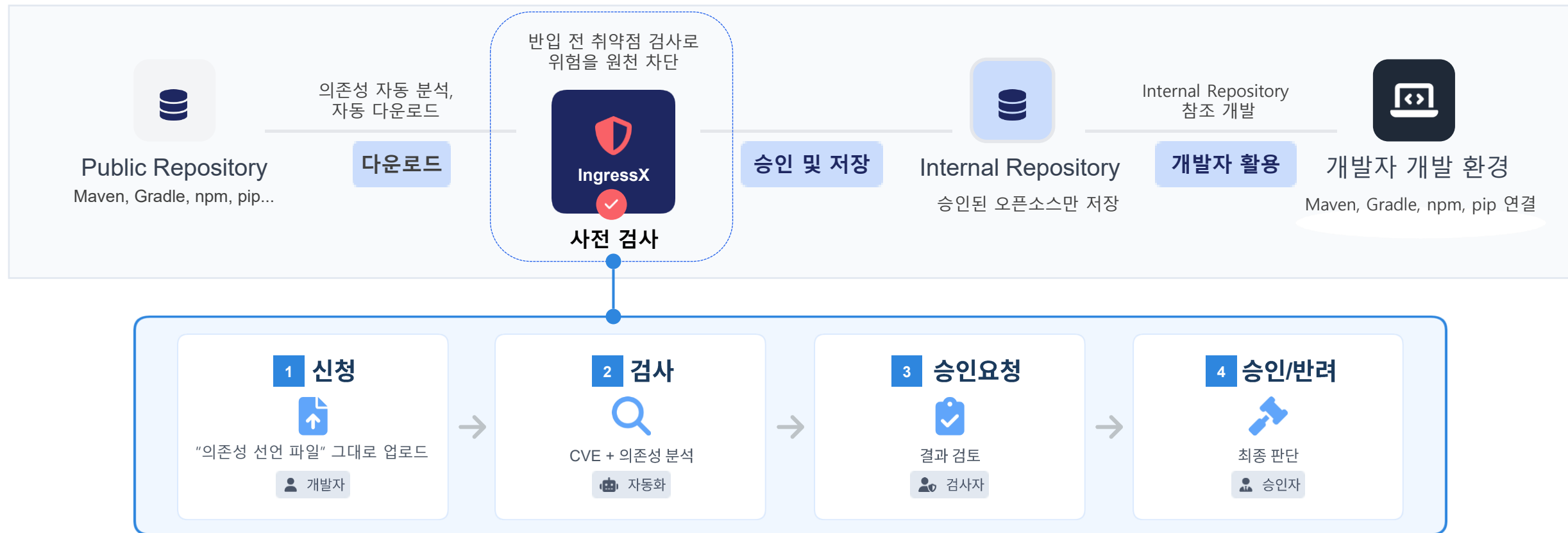
일관성 없는 보안 수준

“

사후 대응이 아닌, **사전 예방 체계**가 필요합니다.

단순 검사뿐 아니라, 신청-검사-승인-관리의 체계적인 통합 솔루션이 필요합니다.

IngressX를 통해 검사 후 승인된 오픈소스만 내부 Repository에서 안전하게 사용합니다



“

IngressX는 외부 오픈소스를 내부 Repository에 반입하기 전에 CVE 기반 취약점을 검사하고, 안전한 오픈소스만 내부에 저장합니다.

반입 전 검사 vs 반입 후 검사, 무엇이 다른가?

반입 후 검사 (기존 방식)

검사 시점 | 이미 코드에 포함된 후

취약점 발견시 | 이미 사용 중

개발자 영향 | 개발 중간에 오픈소스 교체

보안 리스크 | 취약한 오픈소스가 일정기간 내부에 존재

VS

반입 전 검사 (IngressX)

검사 시점 | 내부에 들어오기 전

취약점 발견시 | 사전 유입 차단

개발자 영향 | 사전에 안전한 것만 사용

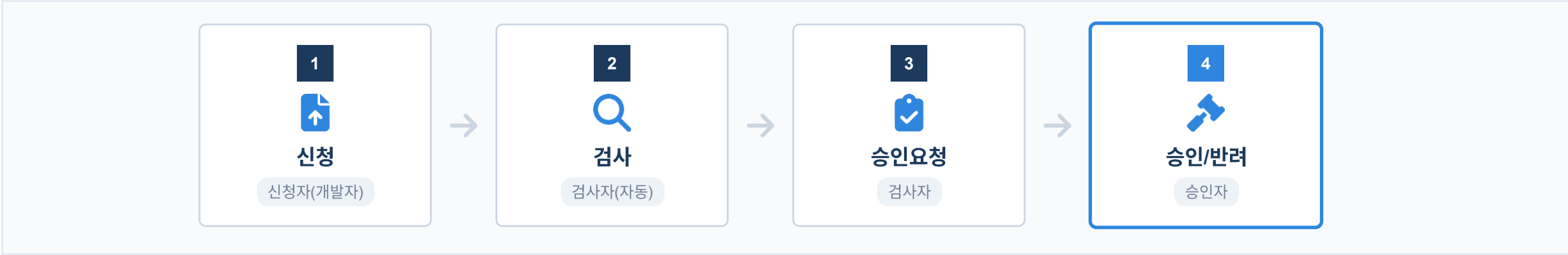
보안 리스크 | 취약한 오픈소스가 내부에 존재하지 않음

“

IngressX는 오픈소스의 "출입국 심사" 체계입니다.

문제가 있는 오픈소스는 **처음부터 내부에 들어오지 못합니다.**

핵심 기능 ① - 체계적인 표준화 프로세스



단계	수행자	행위	결과
① 신청	신청자(개발자)	pom.xml 등 프로젝트 “의존성 선언 파일” 업로드	신청 접수
② 검사	검사자	검사 실행 → 자동 분석/다운로드/CVE 스캔	검사 결과 리포트 생성
③ 승인요청	검사자	검사 결과 검토 후 승인자에게 승인 요청	승인 대기 상태
④ 승인/반려	승인자	검사 결과 기반 최종 판단	Repository 저장

“ 신청-검사-승인의 체계적 프로세스
책임 소재가 명확한 거버넌스(Governance)를 제공합니다.

핵심 기능 ② - 간편한 신청



개발자 입력

개발자가 프로젝트에서 사용중인
"의존성 선언 파일"을 그대로 업로드
합니다.

pom.xml (Maven)

build.gradle (Gradle)

package.json (npm)

requirements.txt (pip)

→ 업로드만 하면 끝

별도의 파일 변환, 가공이나
로컬 CLI 도구 설치가 불필요합니다.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>2.0.0.RELEASE</version>
  <scope>compile</scope>
</dependency>
```

pom.xml

개발자가 프로젝트에서 사용중인
"의존성 선언 파일"을 그대로 업로드

검사 신청

프로젝트명

홈페이지

파일 업로드 ⓘ

Choose File

No file chosen

업로드

무결성 검증 해시

SHA-256 ▾

예) 3f786850e...

업로드 목록

번호	파일명	파일 종류	파일 크기(KB)	사전 검사	중복 신청 여부	삭제
1	pom.xml	패키지 파일	1,016	✓ 정상	미중복	🗑

작성 취소

검사 신청

“

새로운 도구를 배울 필요가 없습니다.

이미 사용 중인 "의존성 선언 파일" 하나면 충분합니다.



자동 의존성 분석

Direct(직접), Transitive(전이) 의존성
까지 자동으로 분석 및 해석

- ▶ Direct Dependencies (직접)
- ▶ Transitive Dependencies (전이/간접)
- ▶ 전체 의존성 분석

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>2.0.0.RELEASE</version>
  <scope>compile</scope>
</dependency>
```

1개의 오픈소스 라이브러리 신청시
전이(간접) 의존성 분석을 통해
수십개의 의존성 라이브러리 확인

```
drwxr-xr-x 4 techware techware 4096 Feb 19 14:16 io
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 ch
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 net
drwxr-xr-x 4 techware techware 4096 Feb 19 14:16 javax
drwxr-xr-x 11 techware techware 4096 Feb 19 14:16 org
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 junit
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 classworlds
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-cli
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-collections
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-lang
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 oro
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-io
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-validator
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-beanutils
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 commons-digester
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 xml-apis
drwxr-xr-x 4 techware techware 4096 Feb 19 14:16 asm
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 backport-util-concurrent
drwxr-xr-x 3 techware techware 4096 Feb 19 14:16 log4j
drwxr-xr-x 4 techware techware 4096 Feb 19 14:16 commons-logging
drwxr-xr-x 4 techware techware 4096 Feb 19 14:16 com
```

```
--- maven-dependency-plugin:2.8:tree (default-cli) @ spring ---
com.techware.test:spring:jar:1.0-SNAPSHOT
\-- org.springframework.boot:spring-boot-starter-web:jar:2.0.0.RELEASE:compile
    +- org.springframework.boot:spring-boot-starter:jar:2.0.0.RELEASE:compile
    | +- org.springframework.boot:spring-boot:jar:2.0.0.RELEASE:compile
    | +- org.springframework.boot:spring-boot-autoconfigure:jar:2.0.0.RELEASE:compile
    | +- org.springframework.boot:spring-boot-starter-logging:jar:2.0.0.RELEASE:compile
    | | +- ch.qos.logback:logback-classic:jar:1.2.3:compile
    | | | +- ch.qos.logback:logback-core:jar:1.2.3:compile
    | | | \-- org.slf4j:slf4j-api:jar:1.7.25:compile
    | +- org.apache.logging.log4j:log4j-to-slf4j:jar:2.10.0:compile
    | | \-- org.apache.logging.log4j:log4j-api:jar:2.10.0:compile
    | \-- org.slf4j:jul-to-slf4j:jar:1.7.25:compile
    +- javax.annotation:javax.annotation-api:jar:1.3.2:compile
    +- org.springframework:spring-core:jar:5.0.4.RELEASE:compile
    | \-- org.springframework:spring-jcl:jar:5.0.4.RELEASE:compile
    \-- org.yaml:snakeyaml:jar:1.19:runtime
+- org.springframework.boot:spring-boot-starter-json:jar:2.0.0.RELEASE:compile
| +- com.fasterxml.jackson.core:jackson-databind:jar:2.9.4:compile
| | +- com.fasterxml.jackson.core:jackson-annotations:jar:2.9.0:compile
| | \-- com.fasterxml.jackson.core:jackson-core:jar:2.9.4:compile
| +- com.fasterxml.jackson.datatype:jackson-datatype-jdk8:jar:2.9.4:compile
| +- com.fasterxml.jackson.datatype:jackson-datatype-jsr310:jar:2.9.4:compile
| \-- com.fasterxml.jackson.module:jackson-module-parameter-names:jar:2.9.4:compile
+- org.springframework.boot:spring-boot-starter-tomcat:jar:2.0.0.RELEASE:compile
| +- org.apache.tomcat.embed:tomcat-embed-core:jar:8.5.28:compile
| +- org.apache.tomcat.embed:tomcat-embed-el:jar:8.5.28:compile
| \-- org.apache.tomcat.embed:tomcat-embed-websocket:jar:8.5.28:compile
+- org.hibernate.validator:hibernate-validator:jar:6.0.7.Final:compile
| +- javax.validation:validation-api:jar:2.0.1.Final:compile
| +- org.jboss.logging:jboss-logging:jar:3.3.0.Final:compile
| \-- com.fasterxml.classmate:jar:1.3.1:compile
+- org.springframework:spring-web:jar:5.0.4.RELEASE:compile
| \-- org.springframework:spring-beans:jar:5.0.4.RELEASE:compile
\-- org.springframework:spring-webmvc:jar:5.0.4.RELEASE:compile
    +- org.springframework:spring-aop:jar:5.0.4.RELEASE:compile
    +- org.springframework:spring-context:jar:5.0.4.RELEASE:compile
    \-- org.springframework:spring-expression:jar:5.0.4.RELEASE:compile
```

자동
다운로드



자동 다운로드

필요한 모든 오픈소스를 자동으로
다운로드 수행

- ▶ 필요한 모든 패키지 수집
- ▶ 버전 정보 정확히 매칭

→ 휴먼 에러 방지

사람이 놓치기 쉬운 의존성 누락 및
검사 누락을 원천적으로 방지합니다.

66

개발자가 보지 못하는 숨겨진 의존성까지
IngressX가 자동으로 찾아내고 검사합니다.

왜 다중 검사 도구를 사용하나?

단일 검사 도구로는 놓치는 취약점 존재 → 상호 보완으로 탐지 범위 확대


단일 검사 도구

→ 
다중 검사 도구



Aqua Security

범용 보안 스캔

Apache 2.0 라이선스

GHSA, NVD 등 소스 활용



Anchore

취약점 전문 스캔

Apache 2.0 라이선스

GHSA, NVD 등 소스 활용



상용 SW 검사 도구

상용 SW 검사 도구
추가 연동 가능

※ 별도 협의 후 연동

“

각각 DB를 가진 검사도구들이 상호 보완하여 검사 사각지대를 최소화 합니다.

다중 검사 도구들의 교차 검사로 놓치는 취약점을 최소화합니다.

명확한 검사 결과를 제공합니다

항 목	설 명
CVE ID	국제 표준 취약점 식별자 (예: CVE-2021-44228)
위험도 점수	CVSS 기반 심각도 점수 (0.0 ~ 10.0점)
심각도 등급	5단계 분류 체계 (Critical / High / Medium / Low / None)
영향 오픈소스	취약점이 존재하는 정확한 오픈소스명 및 버전
조치 권장 버전	취약점을 제거하기 위해 최소한으로 업그레이드해야 하는 안전 버전

보안 검사 현황							
번호	검사명	검사 종류	검사 상태	취약점 검출	검사 시작 일시	검사 종료 일시	검사 결과 다운로드
1	Syft	SBOM 추출	검사 완료	해당 없음	2026-02-19 14:28:58	2026-02-19 14:29:00	↓ JSON 다운로드
2	Trivy	Trivy 파일 보안 검사	검사 완료	미검출	2026-02-19 14:29:02	2026-02-19 14:29:03	↓ JSON 다운로드
3	Grype	Grype 파일 보안 검사	검사 완료	검출	2026-02-19 14:29:01	2026-02-19 14:29:07	↓ JSON 다운로드

취약점 목록						
취약점 ID	심각도 ①	심각도 점수 ①	패키지명	패키지 버전	조치 권장 버전	취약점 설명
CVE-2018-14721	CRITICAL	10.0	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.7	Server-Side Request Forgery (SSRF) in jackson-databind
CVE-2016-1000027	CRITICAL	9.8	org.springframework:spring-web	5.0.4.RELEASE	6.0.0	Pivotal Spring Framework contains unsafe Java deserialization methods
CVE-2018-11307	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.6	Deserialization of Untrusted Data in jackson-databind
CVE-2018-14718	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.7	Arbitrary Code Execution in jackson-databind
CVE-2018-14719	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.7	Arbitrary Code Execution in jackson-databind
CVE-2018-14720	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.7	XML External Entity Reference (XXE) in jackson-databind
CVE-2018-19360	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.8	Deserialization of Untrusted Data in jackson-databind due to polymorphic deserialization
CVE-2018-19361	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.8	Deserialization of Untrusted Data in jackson-databind
CVE-2018-19362	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.8	com.fasterxml.jackson.core:jackson-databind vulnerable to Deserialization of Untrusted Data
CVE-2018-7489	CRITICAL	9.8	com.fasterxml.jackson.core:jackson-databind	2.9.4	2.9.5	FasterXML jackson-databind allows unauthenticated remote code execution

“ 취약점 정보, 어려워하지 마세요.
한눈에 파악할 수 있습니다.



Internal Repository 재검사

신규 CVE 공개시 영향을 받는 오픈소스
즉시 탐지

- ▶ 관리자 수동 재검사 실행
- ▶ 시스템 스케줄 재검사 실행



운영 시나리오 예시

Event → 긴급 CVE 공지

Action 1 → 재검사 실행 → 영향 패키지 식별

Action 2 → 승인 재검토 / 사용 제한 / 교체 계획

→ 지속적 보안관리

이미 내부로 유입된 오픈소스에 대해서도
지속적인 보안 관리를 유지합니다.



CVE 공개

신규 취약점 정보 발표



재검사 실행

내부 Nexus 전수 검사



영향 목록

사용 중인 취약 패키지 식별



조치

사용 중지 / 업데이트 / 예외

1 표준화 프로세스 제공

개발자의 Public Repository 직접 연결을 차단하고 개발자 파일을 그대로 활용하여 **반입 신청, 자동 다운로드, 검사, 승인** 등 WEB UI 기반의 편리한 **표준화 프로세스**를 제공합니다.

2 반입 전 사전 검사

외부 오픈소스를 내부로 **반입하기 전에 취약점을 검사**하고, 안전한 오픈소스만 Internal Repository에 저장합니다. 문제가 있는 오픈소스는 처음부터 들어오지 못합니다.

3 개발자 파일 그대로 활용

pom.xml (Maven), build.gradle (Gradle), package.json (npm), requirements.txt (pip) 등의 **개발자가 사용중인 "의존성 선언 파일"을 그대로 업로드**하여 반입을 신청합니다. "의존성 선언 파일" 하나면 충분합니다.

4 다중 검사 도구

다중 검사 도구를 활용한 취약점 검사를 진행합니다. 다중 검사 도구로 상호 보완하여 탐지 사각지대를 최소화합니다. 별도 협의 후 상용 SW 검사 도구 추가 연동이 가능합니다.

5 유연한 Internal Repository

승인된 오픈소스를 Internal Repository에 저장하여 **조직 표준 Repository로 운영**합니다. Internal Repository를 용도와 목적에 따라 분리하여 운영이 가능합니다.

6 지속적 관리 및 재검사

이미 내부로 유입된 오픈소스에 대해서도 **주기적 재검사로 지속적인 취약점 대응**을 유지합니다. 수동 재검사 실행과 스케줄 재검사 실행으로 신규 CVE 공개시 영향 받는 오픈소스를 즉시 탐지합니다.

항 목	A 제품	B 제품	IngressX
검사 시점	개발 중/후	개발 중/후	✓ 개발 전 그리고 중/후
워크플로우	수동	수동	✓ 표준화
승인 체계	별도 구성	별도 구성	✓ 기본 제공
의존성 분석	제한적	제한적	✓ 자동화
검사 도구	단일	단일	✓ 다중화
Internal Repository	미제공	미제공	✓ 기본 제공 + 추가(옵션)
초기 투자 비용	높음	매우 높음	✓ 매우 낮음
운영 복잡도	높음	매우 높음	✓ 매우 낮음

“ 경쟁 제품들은 "반입 후 검사"가 기본입니다.
IngressX는 반입 전부터 검사하여 지속적으로 취약점 관리가 가능합니다.

도입 전

AS-IS

- ✗ 개발자가 **Public Repository**에서 직접 다운로드 하거나, **개발 소스에 수동 직접 포함**
- ✗ 취약점 검사 없이 사용
- ✗ 취약점 발견은 **개발 중/후**
- ✗ 이미 침투한 취약점 제거 어려움



도입 후 with IngressX

TO-BE

- ✓ 개발자의 **Public Repository** 직접 접근 차단하여 취약한 오픈소스의 내부 유입 차단
- ✓ 취약점 검사 다중화
- ✓ 취약점 발견은 **개발 전** 그리고 **중/후**
- ✓ 취약한 오픈소스 반입전 차단 + 주기적 재검사

실질적 효과



Performance

취약점 차단

- 반입 전 검사로 사전 차단
- 알려진 취약점 즉시 탐지
- 주기적 재검사로 지속적 취약점 관리



Defense

공급망 공격 방어

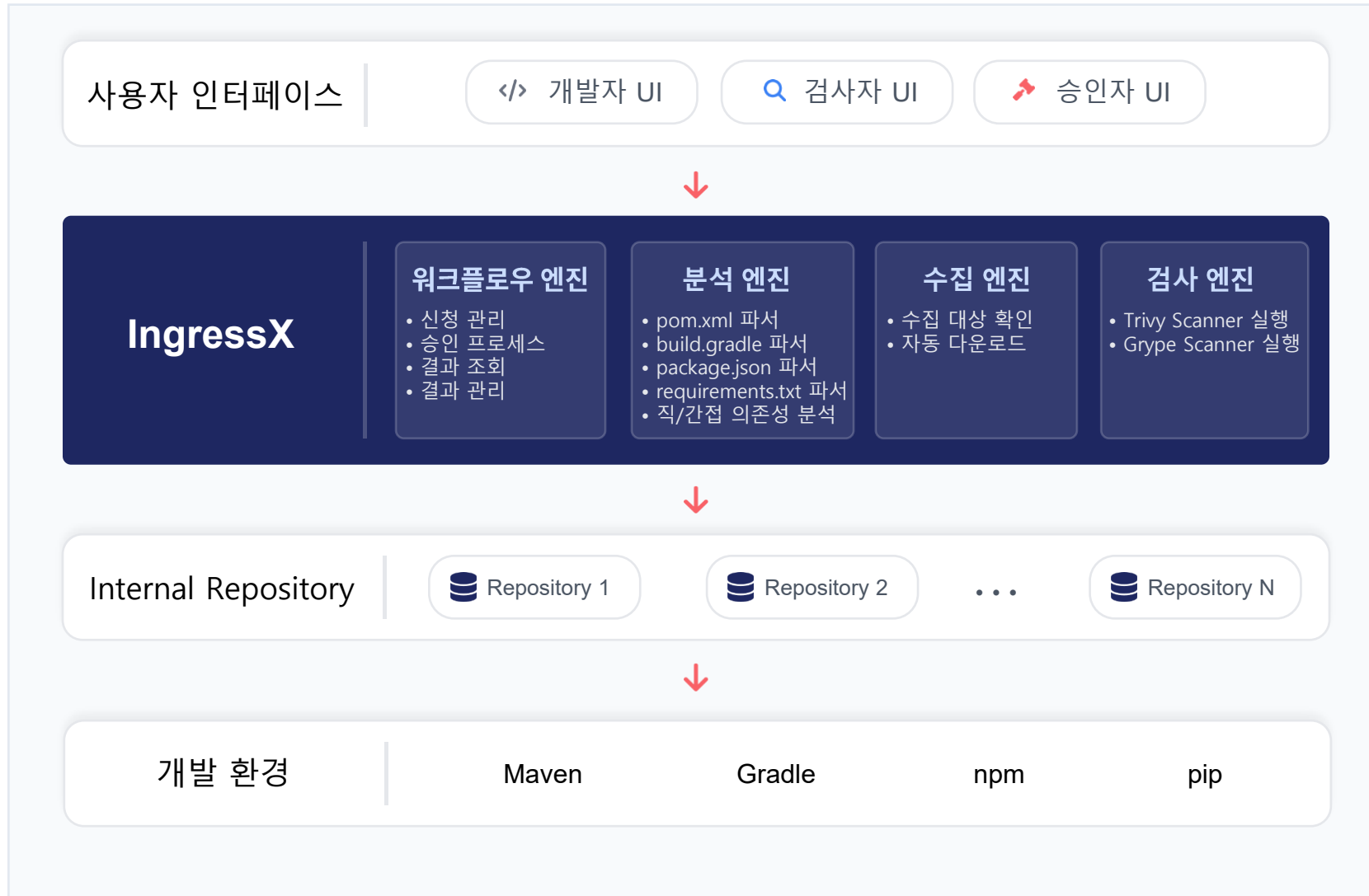
- 의존성 혼동 방지
- 타이포스쿼팅 방지



Audit

감사 대응

- 모든 오픈소스 반입 이력 기록
- 승인 근거와 책임 소재가 명확



지원 오픈소스 생태계

pom.xml (Maven)

build.gradle (Gradle)

package.json (npm)

requirements.txt (pip)

지원 검사 도구



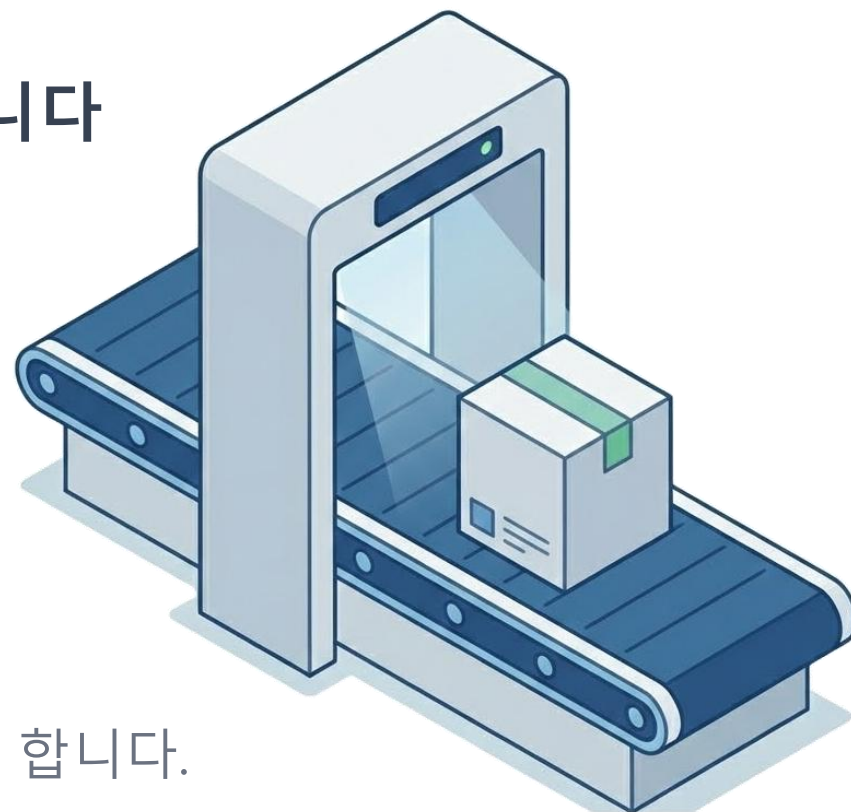
+



검사 및 승인되지 않은 오픈소스는 들어오지 못합니다

IngressX

Public Repository 직접 사용을 차단하고,
승인된 오픈소스만 Internal Repository에 반입하여 사용하게 합니다.



Email

techware@techware.co.kr



전화

042-489-7515



홈페이지

www.IngressX.co.kr